DATABASE MANAGEMENT SYSTEMS
ASSIGNMENT 1 – CO1

DESIGN, ANALYSE AND VALIDATE A STORAGE AND INDEXING STRATEGY FOR THE MUSEUM CONDITION-REPORT DATABASE

Submitted by: p.shalini

Register Number: (1965260185)

B.Tech CSE (Cybersecurity)

CSA05 – Database Management Systems

Introduction and assumptions

This answer is based on the Meridian Museum scenario in the supplied assignment. The design keeps four facts separate: legal ownership, temporary loan custody, exhibition placement and condition evidence. The preferred core uses independently identified LOAN records, LOAN_OBJECT lines for each artifact, and CONDITION_REPORT records linked to the physical artifact.

The assignment asks for actual MySQL outputs and measured query timings. The SQL below is reproducible, but it has not been executed on the user's computer. Therefore, no invented output or benchmark is presented. Run the queries and insert the actual rows, row counts and timings. SQL uses MySQL 8.0 syntax; MySQL 5.7 does not enforce CHECK constraints.

TASK 1 – Requirements and alternative conceptual designs

Requirements

- Registrars create loan and custody records; curators plan exhibitions and placements; conservators create condition reports; collection staff maintain artifact and lender identities.
- Shared data includes artifact identity, lender, agreement number, loan start/due/return dates, exhibition/slot, report date, condition observations and responsible employee.
- Ownership is legal title; custody is temporary physical possession; placement is where an artifact is displayed; condition evidence is an observation about the object at a time.
- Condition reports must identify the artifact, not merely the lender. An artifact may have successive loans and reports.

Alternative A – Dated relationship

```
LENDER  ──< BORROWS(start_date, due_date, returned_date) >── ARTIFACT

ARTIFACT ──< CONDITION_REPORT >── CURATOR
```

The dated LENDER–ARTIFACT relationship itself represents a custody period. It is compact and preserves dates, but a single agreement containing many artifacts repeats agreement data and has no independent loan identity. It resolves the case where reports were filed by lender name by making the report point to the artifact.

Alternative B – Independent LOAN occurrence (preferred)

```
LENDER 1 ──< LOAN 1 ──< LOAN_OBJECT >── 1 ARTIFACT

ARTIFACT 1 ──< CONDITION_REPORT (optionally references LOAN_OBJECT)
```

LOAN has its own loan_id. LOAN_OBJECT identifies each artifact within the agreement and stores item-level received/returned dates and valuation. It supports several artifacts in one agreement and successive loans for one artifact without overwriting history. It solves the failure of a single artifact-level current-loan field.

Alternative C – Event/history model

```
LOAN 1 ──< CUSTODY_EVENT >── 1 ARTIFACT

ARTIFACT 1 ──< CONDITION_REPORT

ARTIFACT 1 ──< PLACEMENT_PERIOD >── DISPLAY_SLOT
```

Custody movements and placements are recorded as dated events. This is best for detailed movements and audit trails, but has more tables and requires event-order and overlap validation.

Alternative	Identity and evidence ownership	Main limitation
A	Dated relationship instance; evidence	Multi-artifact agreement is awkward

	linked to artifact and custody relationship	
B	Independent LOAN; LOAN_OBJECT owns loan-line facts; report belongs to artifact	Extra events needed for detailed internal moves
C	Custody events and placement periods preserve full history	More complex queries and validation

Renaming BORROWS to LOAN does not create a new model. The alternatives differ in identity: a dated relationship, an independent loan entity, or a sequence of events. Alternative B is selected as the best balance for the stated multi-artifact loan rules.

TASK 2 – Entities, attributes and keys

Entity	Primary key / candidate key	Notes
PERSON	person_id; email only if verified unique	Name components and contact details
ORGANIZATION	organization_id; registration_no	Legal identity
LENDER	lender_id; lender_code	Links to person or organization
EMPLOYEE	employee_id; person_id unique	Role and hire date
CURATOR / REGISTRAR	employee_id	Employee subtypes
ARTIFACT	artifact_id; accession_no unique	Title, description, CollectionType
LOAN	loan_id; agreement_no unique	One coordinating lender
LOAN_OBJECT	(loan_id, artifact_id)	Artifact line in a loan
EXHIBITION	exhibition_id	Exhibition name is not necessarily unique
DISPLAY_SLOT	(exhibition_id, slot_no)	Weak, owner-dependent entity
CONDITION_REPORT	report_id	Artifact, date, stage, observations
ARTIFACT_OWNER	(artifact_id, owner_party_id, valid_from)	Ownership share and validity dates

DISPLAY_SLOT depends on EXHIBITION; slot_no is only locally unique. An exhibition name cannot identify a slot because names can repeat or change and slot numbers restart in each exhibition. Use a stable generated primary key for strong entities and UNIQUE constraints for business identifiers. Concurrent inserts are protected by PRIMARY KEY/UNIQUE indexes; a duplicate registration is rejected. Names are unreliable because different people can share a name or change it.

Phone numbers are multivalued; CollectionType is a code/domain unless it has independent data; source dates are stored attributes; report counts and custody-day totals are derived summaries.

TASK 3 – Relationships and constraints

Relationship	Participation bounds	Reason
LENDER–LOAN	LENDER 0..N; LOAN exactly 1	Lender may have no loans; each loan has one coordinating lender
LOAN–LOAN_OBJECT	LOAN 1..N; line exactly 1	An approved loan contains one or more artifact lines
ARTIFACT–LOAN_OBJECT	ARTIFACT 0..N; line exactly 1	Artifact may never be loaned or may have successive loans
ARTIFACT–CONDITION_REPORT	ARTIFACT 0..N; report exactly 1	New artifact may have no report; each report identifies one artifact
EXHIBITION–DISPLAY_SLOT	EXHIBITION 0..N; slot exactly 1	Slot depends on its exhibition

Placement must reference the composite key (exhibition_id, slot_no) of DISPLAY_SLOT. This prevents a placement from pairing an exhibition with a slot owned by another exhibition. A binary LENDER–LOAN link and LOAN–employee creator link are sufficient when the loan is independently identified. A ternary LENDER–CURATOR–EXHIBITION relationship is needed only if the exact three-way fact 'this curator coordinated this lender's contribution to this exhibition' must be recorded.

Agreement start/due dates belong to LOAN; received/returned dates and loan-line valuation belong to LOAN_OBJECT; observations and report date belong to CONDITION_REPORT; curator responsibility for a placement belongs to PLACEMENT or a separate assignment association.

Ternary projection test

Let R contain (L1,C1,E1) and (L1,C2,E2). Binary projections are LC={(L1,C1),(L1,C2)}, LE={(L1,E1),(L1,E2)}, CE={(C1,E1),(C2,E2)}. Joining them produces spurious combinations (L1,C1,E2) and (L1,C2,E1). Thus the binary projections do not reconstruct the ternary relation. A loan with no LOAN_OBJECT line may remain a draft, but must not be activated; this is a cross-table rule.

TASK 4 – Chen ER representation and attribute catalogue

Textual Chen-style ER sketch (1 means one; N means many):

```
[LENDER] 1 ──<HAS>── N [LOAN] 1 ──<CONTAINS>── N [LOAN_OBJECT] N ──<FOR>── 1 [ARTIFACT]

[ARTIFACT] 1 ──<HAS>── N [CONDITION_REPORT]

[EXHIBITION] 1 ──<IDENTIFIES>── N [DISPLAY_SLOT]

[ARTIFACT] 1 ──<PLACED_IN>── N [PLACEMENT] N ──<USES>── 1 [DISPLAY_SLOT]

[EMPLOYEE] 1 ──<CREATES>── N [LOAN]
```

Attribute catalogue: PERSON(person_id, first_name, middle_name, last_name, email); ORGANIZATION(organization_id, legal_name, registration_no); LENDER(lender_id, person_id, organization_id, lender_code); EMPLOYEE(employee_id, person_id, role, hire_date); ARTIFACT(artifact_id, accession_no, title, description, collection_type); LOAN(loan_id, agreement_no, lender_id, created_by_employee_id, start_date, due_date, status); LOAN_OBJECT(loan_id, artifact_id, received_date, returned_date, loan_line_value, dispatch_condition); CONDITION_REPORT(report_id, artifact_id, loan_id, report_date, stage, observations, assessed_by_employee_id); EXHIBITION(exhibition_id, exhibition_name, start_date, end_date); DISPLAY_SLOT(exhibition_id, slot_no, gallery, location_label); PLACEMENT(placement_id, artifact_id, exhibition_id, slot_no, planned_from, planned_to, actual_from, actual_to).

DISPLAY_SLOT is identified by EXHIBITION plus its partial key slot_no. The identifying relationship is EXHIBITION–DISPLAY_SLOT. A foreign key in PLACEMENT is an ordinary reference. Repeated ARTIFACT boxes in a drawing are aliases of the same entity. Double lines indicate mandatory participation; single lines indicate optional participation.

TASK 5 – EER model

```
PERSON
  ├── LENDER_ROLE
  └── EMPLOYEE (role discriminator)
        ├── CURATOR
        └── REGISTRAR
```

LENDER and EMPLOYEE roles overlap: a curator can lend a privately owned object. CURATOR and REGISTRAR are subtypes of EMPLOYEE. The specialization is total only if every employee must be one of these two roles; otherwise it is partial. It is overlapping if an employee may hold both roles; otherwise it is disjoint. A single Role column works only for mutually exclusive roles; multiple roles require a role-membership table.

A notification contact may be a PERSON or ORGANIZATION. Model this as a union category CONTACT_PARTY, or a contact table with exactly one of person_id and organization_id populated. A wide table containing all lender, curator and

registrar fields creates nulls and duplicates. Shared identity belongs in PERSON; role-specific data belongs in subtype tables. Condition reports are assessed by employees; custody is recorded separately in LOAN and LOAN_OBJECT.

TASK 6 – Weak entities and complex attributes

DISPLAY_SLOT is weak: owner EXHIBITION, partial key slot_no, full key (exhibition_id, slot_no). Store phone numbers as PERSON_PHONE(person_id, phone_number, phone_type), one row per number. Split names into first, middle and last components. Artifact appraised value is different from loan_line_value negotiated for one loan.

Current custody days, total report count and current ownership percentage are derived summaries and should normally be calculated. Store source dates and observations. CollectionType can remain a controlled code; if it has its own description, handling rules or relationships, create COLLECTION_TYPE(type_id, type_name, handling_guidance).

TASK 7 – EER to relational schema

- PERSON(person_id PK, first_name, middle_name, last_name, email)
- PERSON_PHONE(person_id PK/FK, phone_number PK, phone_type)
- ORGANIZATION(organization_id PK, legal_name, registration_no UNIQUE)
- LENDER(lender_id PK, person_id FK NULL, organization_id FK NULL, lender_code UNIQUE)
- EMPLOYEE(employee_id PK, person_id FK UNIQUE, role, hire_date); CURATOR(employee_id PK/FK); REGISTRAR(employee_id PK/FK)
- ARTIFACT(artifact_id PK, accession_no UNIQUE, title, description, collection_type)
- LOAN(loan_id PK, agreement_no UNIQUE, lender_id FK, created_by_employee_id FK, start_date, due_date, status)
- LOAN_OBJECT(loan_id PK/FK, artifact_id PK/FK, received_date, returned_date, loan_line_value, dispatch_condition)
- CONDITION_REPORT(report_id PK, artifact_id FK, loan_id nullable, report_date, stage, observations, assessed_by_employee_id FK)
- EXHIBITION(exhibition_id PK, exhibition_name, start_date, end_date); DISPLAY_SLOT(exhibition_id PK/FK, slot_no PK, gallery, location_label)
- PLACEMENT(placement_id PK, artifact_id FK, exhibition_id, slot_no, planned_from, planned_to, actual_from, actual_to, composite FK to DISPLAY_SLOT)

The overlapping PERSON roles map to one PERSON table plus separate LENDER and EMPLOYEE subtype tables; CURATOR and REGISTRAR use employee_id as both PK and FK. Historical loans should not be deleted when evidence exists. Use CLOSED/CANCELLED status and RESTRICT deletion of parent records to preserve custody and condition history.

TASK 8 – MySQL implementation and population

The following core DDL is executable in MySQL 8.0. Add the remaining tables from Task 7 for a complete implementation.

```
CREATE DATABASE IF NOT EXISTS meridian_museum;

USE meridian_museum;

CREATE TABLE person (

 person_id BIGINT PRIMARY KEY AUTO_INCREMENT,

 first_name VARCHAR(60) NOT NULL, middle_name VARCHAR(60),

 last_name VARCHAR(60), email VARCHAR(120) UNIQUE);
```

```
CREATE TABLE organization (
 organization_id BIGINT PRIMARY KEY AUTO_INCREMENT,
 legal_name VARCHAR(150) NOT NULL, registration_no VARCHAR(60) UNIQUE);
CREATE TABLE lender (
 lender_id BIGINT PRIMARY KEY AUTO_INCREMENT,
 person_id BIGINT NULL, organization_id BIGINT NULL,
 lender_code VARCHAR(30) NOT NULL UNIQUE,
 FOREIGN KEY(person_id) REFERENCES person(person_id),
 FOREIGN KEY(organization_id) REFERENCES organization(organization_id));
CREATE TABLE employee (
 employee_id BIGINT PRIMARY KEY AUTO_INCREMENT,
 person_id BIGINT NOT NULL UNIQUE,
 role ENUM('CURATOR','REGISTRAR','OTHER') NOT NULL,
 hire_date DATE, FOREIGN KEY(person_id) REFERENCES person(person_id));
CREATE TABLE artifact (
 artifact_id BIGINT PRIMARY KEY AUTO_INCREMENT,
 accession_no VARCHAR(40) NOT NULL UNIQUE,
 title VARCHAR(180) NOT NULL, description TEXT,
 collection_type VARCHAR(50) NOT NULL);
CREATE TABLE loan (
 loan_id BIGINT PRIMARY KEY AUTO_INCREMENT,
 agreement_no VARCHAR(40) NOT NULL UNIQUE,
 lender_id BIGINT NOT NULL, created_by_employee_id BIGINT NOT NULL,
 start_date DATE NOT NULL, due_date DATE NOT NULL,
 status ENUM('DRAFT','OPEN','CLOSED','CANCELLED') DEFAULT 'DRAFT',
 FOREIGN KEY(lender_id) REFERENCES lender(lender_id),
 FOREIGN KEY(created_by_employee_id) REFERENCES employee(employee_id),
 CHECK(due_date >= start_date));
CREATE TABLE loan_object (
 loan_id BIGINT NOT NULL, artifact_id BIGINT NOT NULL,
 received_date DATE NOT NULL, returned_date DATE NULL,
 loan_line_value DECIMAL(12,2), dispatch_condition VARCHAR(500),
 PRIMARY KEY(loan_id,artifact_id),
 FOREIGN KEY(loan_id) REFERENCES loan(loan_id),
```

```
 FOREIGN KEY(artifact_id) REFERENCES artifact(artifact_id),
 CHECK(returned_date IS NULL OR returned_date >= received_date));
CREATE TABLE condition_report (
 report_id BIGINT PRIMARY KEY AUTO_INCREMENT,
 artifact_id BIGINT NOT NULL, loan_id BIGINT NULL,
 report_date DATETIME NOT NULL,
 stage ENUM('PRE_LOAN','RECEIPT','DURING_LOAN','RETURN','POST_LOAN') NOT NULL,
 observations TEXT NOT NULL, assessed_by_employee_id BIGINT NOT NULL,
 FOREIGN KEY(artifact_id) REFERENCES artifact(artifact_id),
 FOREIGN KEY(assessed_by_employee_id) REFERENCES employee(employee_id),
 FOREIGN KEY(loan_id,artifact_id) REFERENCES loan_object(loan_id,artifact_id));
```

MySQL 5.7 does not enforce CHECK constraints. Use triggers for date validity. A loan must contain at least one LOAN_OBJECT before becoming OPEN; ownership shares must total 100% for each effective date. These are cross-row rules and need a transaction/stored procedure or trigger with locking.

Reproducible population plan

```
SET @seed := 4211;
```

Generate parent records before child records: PERSON/ORGANIZATION, LENDER, EMPLOYEE, ARTIFACT, LOAN, LOAN_OBJECT, CONDITION_REPORT. Use a deterministic generator such as Python random.Random(4211). Select foreign keys from already generated parent IDs; do not invent independent IDs. Keep the schema fixed and run four datasets with 100, 1,000, 10,000 and 100,000 lenders. Record script version, seed, row counts, load duration and integrity checks.

TASK 9 – Use-case validation queries

1. Open overdue loans

```
SELECT loan_id, agreement_no, due_date FROM loan WHERE status='OPEN' AND due_date < CURDATE();
```

2. Artifacts without condition evidence

```
SELECT a.artifact_id,a.accession_no FROM artifact a LEFT JOIN condition_report c ON
c.artifact_id=a.artifact_id WHERE c.report_id IS NULL;
```

3. Artifacts currently on loan

```
SELECT DISTINCT a.artifact_id,a.accession_no,l.agreement_no FROM artifact a JOIN loan_object lo ON
lo.artifact_id=a.artifact_id JOIN loan l ON l.loan_id=lo.loan_id WHERE lo.returned_date IS NULL AND
l.status='OPEN';
```

4. Loan and its artifact lines

```
SELECT l.agreement_no,a.accession_no,lo.received_date,lo.returned_date FROM loan l JOIN loan_object lo
ON lo.loan_id=l.loan_id JOIN artifact a ON a.artifact_id=lo.artifact_id;
```

5. Condition history for an artifact

```
SELECT a.accession_no,c.report_date,c.stage,c.observations FROM artifact a JOIN condition_report c ON
c.artifact_id=a.artifact_id WHERE a.accession_no='ART-0001' ORDER BY c.report_date;
```

6. Loans due within 30 days

```
SELECT loan_id,agreement_no,due_date FROM loan WHERE status='OPEN' AND due_date BETWEEN CURDATE() AND
DATE_ADD(CURDATE(),INTERVAL 30 DAY);
```

7. Reports per artifact

```
SELECT a.accession_no,COUNT(c.report_id) AS report_count FROM artifact a LEFT JOIN condition_report c
ON c.artifact_id=a.artifact_id GROUP BY a.artifact_id,a.accession_no;
```

8. Loans with multiple artifacts

```
SELECT l.agreement_no,COUNT(*) AS artifact_count FROM loan l JOIN loan_object lo ON
lo.loan_id=l.loan_id GROUP BY l.loan_id,l.agreement_no HAVING COUNT(*)>1;
```

Record actual output rows, including '0 rows' for empty results. Run each query several times after warm-up and report the median elapsed time. Use EXPLAIN. Suggested indexes: loan(status,due_date), loan_object(artifact_id,returned_date), condition_report(artifact_id,report_date), plus all PK/FK indexes.

Dataset	Query	Rows returned	Median ms	Notes
100 lenders	1–8	Fill actual values	Measure	Record JOIN count
1,000 lenders	1–8	Fill actual values	Measure	Same schema
10,000 lenders	1–8	Fill actual values	Measure	Same schema
100,000 lenders	1–8	Fill actual values	Measure	Same schema

Flat-file anomalies: in LOAN_FILE(agreement_no,lender_name,lender_phone,artifact_no,due_date,condition_text), lender details repeat for every artifact. Updating only some rows causes an update anomaly; a lender cannot be inserted without a loan (insertion anomaly); deleting the last artifact line may delete the only lender details (deletion anomaly). Separate relations remove these problems.

TASK 10 – File, hierarchical and network comparison

Model	Access path	Evaluation
File-based	Scan separate lender, loan, artifact and report files	Duplication, weak integrity and anomalies
Hierarchical	LENDER -> LOAN -> LOAN_OBJECT -> ARTIFACT tree	Fast for fixed parent-child path; difficult for artifacts in successive loans and multiple placements
Network	Owner/member sets connect lender-loan, loan-loan line and artifact-loan line	Multiple paths supported, but pointer navigation is complex
Relational	PK/FK relations and SQL joins	Flexible queries, declarative constraints and transactions; indexes need tuning

Overdue-loan query: file model scans loan file; hierarchy traverses each lender's loan children; network follows lender-to-loan sets; relational model filters LOAN by status and due_date index. Artifact-without-report query: file model compares files; hierarchy searches report branches; network follows artifact-to-report sets; relational model uses LEFT JOIN and IS NULL.

Three-schema architecture: external level contains registrar, curator and conservator views; conceptual level is the shared logical database; internal level contains pages, indexes and record layouts. Query processor/optimizer plans SQL; transaction and lock managers coordinate concurrent updates; buffer/storage managers handle pages; recovery manager uses logs. Concurrent changes to one loan should be committed in a transaction to avoid lost updates.

TASK 11 – Final recommendation

Choose Alternative B as the core, with event history added only if detailed internal movements are required. It gives LOAN an independent identity, supports multiple artifacts per agreement and preserves successive custody periods in LOAN_OBJECT. CONDITION_REPORT belongs to the physical artifact and may also reference the relevant loan line.

Requirement mapping: lender identity -> PERSON/LENDER; agreement dates -> LOAN; per-artifact custody -> LOAN_OBJECT; display location -> PLACEMENT/DISPLAY_SLOT; condition evidence -> CONDITION_REPORT; legal ownership -> ARTIFACT_OWNER added in Task 12. Query outputs and timings must be filled after real execution; they are not claimed here. At 100,000 lenders, retain surrogate keys, index foreign keys, batch loads and inspect query plans. Cross-table temporal rules remain a limitation requiring transactions or triggers.

TASK 12 – Re-evaluate and adapt the design

A. Impact analysis

The original assumption that an artifact has one current legal owner/lender is invalid. Legal ownership may be shared and shares may change while a loan remains active. LENDER cannot represent ownership by itself. LOAN and LOAN_OBJECT remain valid because they represent custody, not title. CONDITION_REPORT remains linked to the artifact and relevant custody line. The old artifact-to-lender identifier cannot express multiple owners, percentages or effective dates.

At the external level, keep the registrar's loaning-party and custody screen. At the conceptual level, add ownership history. Internally, index artifact_id and validity dates.

B. Evaluate original design

The original single-current-lender representation fails the shared-ownership rule. EXHIBITION is a display context, not an ownership record. Stable keys artifact_id, loan_id and exhibition_id remain unchanged; the ownership participation and temporal constraints must change. Alternative A can date an owner relationship but does not cleanly manage shared percentages. Alternative C can record change events but still needs an ownership-period relation. Alternative B remains the best custody core.

C. Adaptation

Decision: MODIFY. Add OWNER_PARTY and ARTIFACT_OWNER. OWNER_PARTY represents either a person or organization. ARTIFACT_OWNER records each owner, share and effective period.

```
OWNER_PARTY(owner_party_id PK, party_type, person_id NULL, organization_id NULL)

ARTIFACT_OWNER(artifact_id PK/FK, owner_party_id PK/FK, valid_from PK, valid_to NULL, ownership_share)

ARTIFACT 1 ──< ARTIFACT_OWNER >── 1 OWNER_PARTY
```

For each artifact and effective date, active ownership shares must total 100%. A row CHECK can enforce 0 < share <= 100, but cannot ensure the sum across rows or prevent temporal overlaps. Validate changes in one transaction with appropriate locking. Migration: create one OWNER_PARTY for each documented existing owner and insert a 100% ownership row from the documented start date. If no reliable date exists, use a clearly labelled migration date; do not invent history. Preserve all loan, loan-line and report rows unchanged. Add co-owner periods only from verified records.

D. Evidence-based defence (

This query finds artifacts whose active shares do not total 100%:

```
SELECT a.artifact_id, COALESCE(SUM(ao.ownership_share),0) AS total_share

FROM artifact a LEFT JOIN artifact_owner ao
```

```
 ON ao.artifact_id=a.artifact_id AND ao.valid_to IS NULL

GROUP BY a.artifact_id

HAVING total_share <> 100.00;
```

Also test for overlapping ownership periods for the same owner/artifact and verify that every LOAN_OBJECT and CONDITION_REPORT still points to the same artifact after migration. Shared ownership adds owner-history rows, but should not change LOAN, LOAN_OBJECT or CONDITION_REPORT counts. Existing operational views can still show the coordinating lender from LOAN and display legal owners in a separate panel. The extra association adds joins, storage and temporal checks, but prevents legal ownership being confused with temporary custody. This modification directly satisfies the changed requirement.

Conclusion

The final design separates legal ownership, loan custody, exhibition placement and condition evidence. Independent LOAN and LOAN_OBJECT records preserve custody history; CONDITION_REPORT follows the physical artifact; ARTIFACT_OWNER records shared legal title over time. The model supports successive loans and overlapping personal/employee roles without rewriting historical custody records.

References

- Elmasri, R. and Navathe, S. B., Fundamentals of Database Systems, Pearson.
- Silberschatz, A., Korth, H. F. and Sudarshan, S., Database System Concepts, McGraw-Hill.
- Connolly, T. and Begg, C., Database Systems: A Practical Approach to Design, Implementation, and Management, Pearson.
- MySQL 8.0 Reference Manual, https://dev.mysql.com/doc/refman/8.0/en/.